

Technical Report: AirBoard Secure Cross-Platform P2P Clipboard Sync

AirBoard is a decentralized, zero-trust, serverless peer-to-peer (P2P) clipboard synchronization system designed to seamlessly and securely share clipboard data across multiple local devices. Unlike cloud-dependent alternatives, AirBoard operates entirely on the local network (LAN), guaranteeing data privacy, near-instant synchronization, and offline capability.

1. Project Overview & Aim

Objective

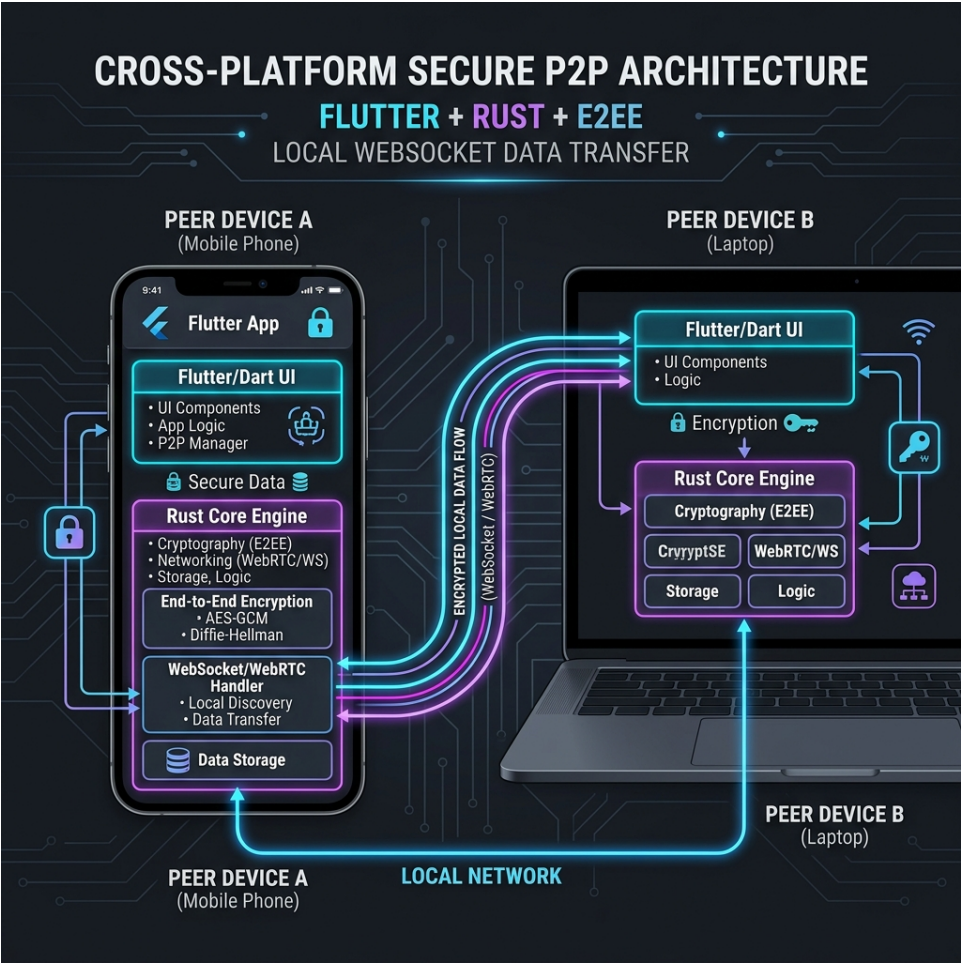
The primary aim of AirBoard is to bridge the clipboard states of heterogeneous operating systems without relying on third-party cloud servers, database coordinators, or central brokers. The system operates on a zero-trust security model, ensuring that clipboard contents are encrypted end-to-end (E2EE) before transit and can only be decrypted by authorized, cryptographically paired peer devices on the same local area network (LAN).

Supported Platforms

AirBoard supports cross-platform execution across major desktop and mobile operating systems: - **Linux** (Debian/Ubuntu, Fedora, Arch) - **Windows** (7/10/11) - **macOS** - **Android** (Phones and Tablets) - **iOS & iPadOS** (iPhones and iPads)

2. System Architecture

AirBoard implements a polyglot architecture combining the graphical capabilities of **Flutter/Dart** with the performance, safety, and speed of **Rust**.



Data Flow Diagram

```
graph TD
    subgraph "Device A (Sender)"
        ClipboardA[System Clipboard] -->|Poll/ChangeCount| DartUI_A[Dart UI Layer]
        DartUI_A -->|FRB Async Call| RustCore_A[Rust Core Engine]
        RustCore_A -->|Encrypt: ChaCha20-Poly1305| Crypto_A[Crypto Module]
        Crypto_A -->|Send Payload| WS_Client[WebSocket Client / Server]
    end

    WS_Client -->|Local LAN / E2EE| WS_Server[WebSocket Server / Client]

    subgraph "Device B (Receiver)"
        WS_Server -->|Receive Payload| Crypto_B[Crypto Module]
        Crypto_B -->|Decrypt| RustCore_B[Rust Core Engine]
        RustCore_B -->|FRB Event Sink| DartUI_B[Dart UI Layer]
        DartUI_B -->|Write| ClipboardB[System Clipboard]
    end
```

Components

- 1. **UI Layer (Flutter/Dart):** Manages the adaptive glassmorphic frontend, monitors local clipboard changes, and handles user configurations (device name, manual connections, trust registers).

2. **Rust Core Engine:** Handles low-level socket networking, peer discovery, cryptographic handshakes, and trust storage.
3. **Bridge Layer (`flutter_rust_bridge`):** A high-performance, zero-copy asynchronous binding layer that handles serialized communication and memory sharing between the Dart runtime and Rust memory space.

3. Core Features

■ Serverless Local Discovery

- Uses **UDP Broadcast** on a dedicated port (45455) to announce device presence on the local network.
- Devices dynamically listen for announcements, parsing IP addresses, ports, and public fingerprints to populate the local "Discovered Devices" register.
- **Manual IP Connection Fallback:** When UDP broadcast packets are blocked (e.g., enterprise WiFi isolations, hotspot restrictions), users can directly initiate pairing by entering the peer's local IP address.

■ End-to-End Encryption (E2EE)

- **Identity Keys:** Each device generates an **Ed25519** signing key pair and an **X25519** Diffie-Hellman key pair upon initial boot, securely stored via platform keychain services.
- **Cryptographic Pairing:** Devices exchange public keys and compute a unique, verifiable SHA-256 fingerprint. Users approve the connection only if fingerprints match on both screens.
- **Session Encryption:** All clipboard payloads are encrypted using **ChaCha20-Poly1305** (AEAD) with unique, cryptographically secure nonces, protecting data from packet sniffing.

■ Optimized Background Execution

- **Android:** Operates a persistent **Foreground Service** with active notification channels to prevent the OS from killing the sync engine. Automatically locks **WiFi Multicast** states to process incoming UDP packets during sleep cycles.
- **iOS & iPadOS:** Employs background keep-alive mechanisms using silent audio loop registers (`AVFoundation`) combined with local notification flags, allowing clipboard buffers to synchronise even when the application is minimized.

■ Adaptive Glassmorphic Interface

- Uses layered translucency mimicking iOS's `UIBlurEffect` with custom CSS-inspired ambient radial backgrounds (blue top-left, purple/teal bottom-right).
- **Responsive Layouts:**
- **Phone (<600px):** Thumb-friendly bottom navigation, consolidated E2EE activity status pill, and tabbed view.

- **Tablet (600px-900px):** Segmented topbar navigator with a collapsible left sidebar.
- **Desktop (>=900px):** Persistent left sidebar separating "Network" and "System" sectors, stats cards row showing system metrics, and structured configuration panels.

4. Technology Stack & Frameworks

Layer	Technology	Purpose
Cross-Platform UI	Flutter (Dart SDK >= 3.3.0)	Cross-platform UI layout rendering & pasteboard access.
Logic & Networking	Rust (Edition 2021)	Multithreaded socket operations, discovery protocols, and file-system registers.
Interoperability	flutter_rust_bridge (v2.12.0)	Zero-copy code generator between Dart and Rust.
Cryptography	ed25519-dalek, x25519-dalek, chacha20poly1305	Key generation, Diffie-Hellman key exchange, and AEAD encryption.
Local Storage	flutter_secure_storage	Cryptographic hardware storage for identity keys (Keychain/Keystore).
Discovery Protocol	UDP sockets	Decentralized local area network peer announcements.
Sync Protocol	WebSockets (tokio-tungstenite)	Reliable, low-latency state synchronisation pipelines.
Platform Integrations	Kotlin (Android), Swift (iOS), C++ GTK (Linux)	Native pasteboard managers, multicast locks, and background execution services.

5. Key Engineering Challenges Solved

1. iOS Clipboard Access Restrictions

- **Challenge:** Apple blocks applications from reading or writing to the system clipboard (Pasteboard) when running in the background.
- **Solution:** Developed a hybrid background architecture. When AirBoard is in the background, incoming clipboard syncs are buffered in memory and a local notification is fired. Upon tapping the notification or

reopening the app, the buffer is flushed to the native clipboard using a custom pasteboard listener.

2. Enterprise Network UDP Isolation

- **Challenge:** Many public and enterprise routers block UDP broadcasting, preventing auto-discovery.
- **Solution:** Designed and implemented a manual IP socket fallback mechanism. By establishing a direct TCP connection via WebSocket handshake to a user-entered IP, devices bypass UDP multicast filters.

3. Native Memory Mapping & Overhead

- **Challenge:** Passing large chunks of clipboard data (formatted text, images, files) between Dart and Rust garbage collectors can lead to memory fragmentation.
- **Solution:** Leveraged Rust's `StreamSink` and Dart's asynchronous streams via zero-copy binary serialization, preventing memory allocation spikes and keeping RAM usage below **40MB** even during large synchronisations.